



Procesadores del Lenguaje
Grado en Ingeniería Informática

Traductor de un subconjunto de C a Lisp

Práctica Final – Procesadores del Lenguaje

Miembros del grupo:

Alejandro Quirante Sanz - 100522183@alumnos.uc3m.es
Carlos Martín Gallardo - 100522258@alumnos.uc3m.es

Grupo 412

2025/2026

1. Introducción

Este proyecto consiste en la implementación de un traductor que convierte código fuente escrito en un subconjunto del lenguaje C a notación prefija de Lisp. La práctica se centra en la construcción de los analizadores léxico y sintáctico, así como en la síntesis de código mediante la propagación de atributos en la gramática.

El sistema es capaz de procesar declaraciones de variables, estructuras de control (`if`, `while`), expresiones aritmético-lógicas y funciones de salida estándar, permitiendo que el resultado sea evaluado directamente por un intérprete de Lisp.

1.1. Jerarquía de Expresiones

Para garantizar la precedencia de operadores, la gramática se ha estructurado en niveles:

expression: Operadores lógicos (`&&`, `||`).

comparison_expression: Operadores relacionales (`==`, `!=`, `<`, `>`, etc.).

additive_expression: Suma y resta.

multiplicative_expression: Multiplicación, división y módulo (`%`).

unary_expression: Operadores de signo (`+`, `-`) y negación (`!`).

2. Generación de Código Lisp

La traducción se basa en la conversión de notación infija de C a la notación prefija de Lisp.

Asignaciones: Una sentencia `a = 5`; se traduce como `(setq a 5)`. Se ha incluido soporte para asignaciones encadenadas del tipo `a = b = 5`; generando estructuras anidadas `(setq a (setq b 5))`.

Estructuras de Control:

`while(cond) { bloque }` se traduce como `(loop while cond do bloque)`.

`if(cond) { bloque }` se traduce como `(if cond bloque)`.

Funciones de Salida:

`puts(x)` se mapea a `(print x)`.

`printf` se ha adaptado para manejar múltiples argumentos, generando una secuencia de llamadas `(print arg)` en Lisp.

3. Pruebas Realizadas

Se utilizó el archivo `prueba.txt` suministrado, el cual contiene casos límite como:

Declaraciones globales múltiples: `int x = 5, y, z = 20;`

Expresiones aritméticas complejas: `temp = a + b - c * d / e;`

Operador módulo y unarios: `temp = -a; temp = a % 3;`

Anidamiento de bucles y condicionales.

El resultado obtenido en el buffer de salida (output_buffer) muestra una estructura de paréntesis balanceada y una traducción fiel de la lógica original.

4. Conclusiones

La práctica ha permitido profundizar en el funcionamiento interno de las herramientas de generación de compiladores. Se ha logrado una traducción eficiente mediante el uso de memoria dinámica para la construcción de cadenas de código, asegurando que la semántica de C se mantenga íntegra tras la conversión a la sintaxis prefija de Lisp.